9.1. Statistical Methods in Data

Previous Section:

9. Exploratory Data Analysis

Contents:

Statistical analysis is the foundation of exploratory data analysis (EDA). It allows us to summarize data, measure its variability, identify patterns, and better understand the characteristics of a dataset before performing visualization or machine learning.

In this section, we will generate a **synthetic student dataset** containing student information and GPA scores. We will then use **Pandas** and **NumPy** to calculate the most common descriptive statistics.

To obtain a quick overview of the dataset, Pandas provides two very useful methods:

- `df.info()` displays the dataset structure, including the number of rows, columns, data types, and missing values.
- `df.describe()` summarizes numerical columns by reporting statistics such as count, mean, standard deviation, minimum, quartiles, and maximum.

Together with NumPy, these libraries provide straightforward functions for computing statistical values efficiently.

```
import numpy as np
import pandas as pd
```

```
np.random.seed(42)

n = 20

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

df = pd.DataFrame({
    "Name": names,
    "Age": np.random.randint(18, 22, n),
    "Major": np.random.choice(["AI", "CSC", "IT", "SW"], n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

df.info()
print(df.describe())
# This extends from df.describe() to return
# statistical summary for every column
print(df.describe(include='all'))
```

1. Basic Statistical Operations

Before studying descriptive statistics such as the mean and median, it is useful to become familiar with the basic statistical operations provided by Pandas. These functions allow us to perform arithmetic on columns, summarize numerical data, and identify the largest or smallest values.

- First, create a second GPA column that will be used to demonstrate arithmetic operations.

```
df["GPA2"] = np.round(np.random.uniform(2.0, 4.0, len(df)), 2)
df[["Name", "GPA", "GPA2"]].head()
```

- **Arithmetic Operations:** Pandas supports element-wise arithmetic between columns.

```
# Addition
df["GPA_Add"] = df["GPA"].add(df["GPA2"])

# Subtraction
df["GPA_Sub"] = df["GPA"].sub(df["GPA2"])

# Multiplication
df["GPA_Mul"] = df["GPA"].multiply(df["GPA2"])

# Division
df["GPA_Div"] = df["GPA"].divide(df["GPA2"])

print(df[["GPA", "GPA2", "GPA_Add", "GPA_Sub", "GPA_Mul", "GPA_Di
v"]].head())
```

- **Maximum and Minimum:** The maximum and minimum values can be obtained using `max()` and `min()`.

```
print("Maximum GPA:", df["GPA"].max())
print("Minimum GPA:", df["GPA"].min())
```

- **Largest and Smallest Values:** To retrieve the largest or smallest observations, use `nlargest()` and `nsmallest()`.

```
# Top 5 highest GPAs
print(df.nlargest(5, "GPA"))

# Bottom 5 lowest GPAs
print(df.nsmallest(5, "GPA"))
```

- **Counting Values:** The `count()` function counts the number of non-missing values in each column.

```
print(df.count())
```

- To count the occurrences of each unique value, use `value_counts()`.

```
print(df["Major"].value_counts())
```

- The same method can also be applied to numerical columns.

```
print(df["Age"].value_counts())
```

- **Summation:** The `sum()` function computes the total of a numerical column.

```
print("Total GPA:", df["GPA"].sum())
```

- It can also be applied to multiple numerical columns.

```
print(df[["GPA", "GPA2"]].sum())
```

1. Mean, Median, and Mode

The most common measures of central tendency are the **mean**, **median**, and **mode**.

- **Mean:** The arithmetic average of all values.
- **Median:** The middle value after sorting the data.
- **Mode:** The value that appears most frequently.

Pandas provides built-in methods to compute each statistic.

```
print("Mean GPA:", df["GPA"].mean())
print("Median GPA:", df["GPA"].median())
print("Mode GPA:")
print(df["GPA"].mode())
```

NumPy also provides equivalent functions.

```
print(np.mean(df["GPA"]))
print(np.median(df["GPA"]))
```

2. Quartiles

Quartiles divide a sorted dataset into four equal parts.

- **Q1 (Lower Quartile):** The median of the lower half of the data.
- **Q2 (Median):** The median of the entire dataset.
- **Q3 (Upper Quartile):** The median of the upper half of the data.

These values are commonly used to understand the spread of data and are essential for box plots and outlier detection.

```
print("Q1:", df["GPA"].quantile(0.25))
print("Q2:", df["GPA"].quantile(0.50))
print("Q3:", df["GPA"].quantile(0.75))
```

We can also simply use: df["GPA"].describe()

3. Variance and Standard Deviation

While measures of central tendency describe the center of the data, **variance** and **standard deviation** describe how spread out the values are.

Since we usually work with **sample data** instead of the entire population, Pandas computes the **sample variance** and **sample standard deviation** by default using **Bessel's correction**, where the denominator is **n – 1** rather than **n**.

```
print("Variance:", df["GPA"].var())
print("Standard Deviation:", df["GPA"].std())
```

NumPy defaults to population variance and standard deviation. To match Pandas, specify `ddof=1`.

```
print(np.var(df["GPA"], ddof=1))
print(np.std(df["GPA"], ddof=1))
```

4. Lambda Function

Lambda functions provide a convenient way to apply simple operations to every value in a Series without defining a separate function.

- For example, we can convert GPA scores to percentages.

```
df["GPA (%)"] = df["GPA"].map(lambda x: round(x / 4 * 100, 1))
print(df.head())
```

- We can also classify students according to their GPA.

```
df["Performance"] = df["GPA"].map(
    lambda x: "Excellent" if x >= 3.5
    else "Good" if x >= 3.0
    else "Average"
)

print(df.head())
```

---

5. Using a User-Defined Function with `map()`

While lambda functions are convenient for simple operations, defining a regular function is often more readable when the logic becomes more complex or when the function will be reused. The function can then be passed directly to `map()`.

- In the example below, we define a function that converts a GPA score into a letter grade.

```
def get_grade(gpa):
    if gpa >= 3.7:
        return "A"
    elif gpa >= 3.3:
        return "B+"
    elif gpa >= 3.0:
        return "B"
    elif gpa >= 2.5:
        return "C"
    else:
        return "D"

df["Letter Grade"] = df["GPA"].map(get_grade)
```

```
print(df[["Name", "GPA", "Letter Grade"]].head())
```

- The same function can easily be reused on another column.

```
df["Letter Grade 2"] = df["GPA2"].map(get_grade)

print(df[["GPA", "GPA2", "Letter Grade", "Letter Grade 2"]].head
())
```

Using a user-defined function makes the code easier to maintain, especially when the same transformation needs to be applied to multiple columns or datasets.

Summary

In this section, we introduced the fundamental statistical methods used in exploratory data analysis. Specifically, we learned how to:

- Use `df.info()` to inspect the structure of a dataset, including data types and missing values.
- Use `df.describe()` to obtain a statistical summary of numerical data.
- Perform basic statistical operations such as addition, subtraction, multiplication, division, counting, summation, and finding maximum and minimum values.
- Retrieve the largest and smallest observations using `nlargest()` and `nsmallest()`.
- Calculate measures of central tendency, including the **mean**, **median**, and **mode**.
- Compute **quartiles** to better understand the distribution of data.
- Measure data variability using **variance** and **standard deviation**.
- Transform data using **lambda functions** together with `map()`.
- Define **user-defined functions** and apply them with `map()` to perform reusable data transformations.

These statistical methods provide a strong foundation for understanding datasets and prepare us for the more advanced exploratory data analysis techniques introduced in the following sections.

References

https://pandas.pydata.org/docs/getting_started/intro_tutorials/06_calculate_statistics.html

https://www.geeksforgeeks.org/python/use-pandas-to-calculate-statistics-in-python/

Next Section:

- 9.2. Data Distribution, Skewness, and Kurtosis